

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\ a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-a773faee7d47d6d6f.jsonl`  
- SHA-256: `0f10547127989c8b13cc94805c07278c17355f0d2c50ffde4870279e916eb831`  
- Source modified: `2026-05-31T04:18:29+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T04:17:42.236Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMv3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON

transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:

****URL:**** <https://lidos.com/en/ai-developer-tools-series-2-open-source-document-parsing.html>

****Title:**** 12 Open-Source PDF Parsing & OCR Tools Evaluated ? MinerU, PaddleOCR & More (Meursault)

****Found via:**** OCR + table-structure recognition pipelines (practitioner/implementation) search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:17:44.261Z)

I'll start by loading the WebFetch tool schema, then retrieve the page content.

3. assistant (2026-05-31T04:17:46.853Z)

I'll fetch the page now.

4. user (2026-05-31T04:18:05.905Z)

PDF/OCR Tools for Financial Tables: Claims & Licenses

Article Metadata

- ****Publish Date:**** December 8, 2024
- ****Author:**** Meursault (?????)
- ****Type:**** Summary/roundup article combining prior benchmarking (Guangzhou Institute of Software Application Technology evaluation) with author's own testing of commercial services

Key Findings on Complex Tables

****General Limitation (All Open-Source Tools):****

> "Most projects support poor performance when processing complex tables and formula recognition."

The article states: "Although there are many excellent open-source projects, they mostly perform poorly when processing complex tables and formula recognition."

Tool-by-Tool Analysis

****MinerU****

- ****Strengths:**** Multi-column layout support; extracts tables to HTML format
- ****Weaknesses:****
 - > "Row and column errors may occur when processing complex tables"

Does not support vertical text; "Comic books, art albums, elementary school textbooks, and exercise books cannot be accurately parsed currently"

- ****License:**** Not stated in article
- ****Hardware Requirements:**** Not stated; article notes it "consumes more computing resources"

****PaddleOCR****

- ****Strengths:**** Layout analysis, table recognition (SLANet_Plus model), multilingual
- ****Weaknesses:**** "Lacks necessary post-processing steps"; no HTML output mentioned
- ****License:**** Not stated in article
- ****Model:**** Based on PaddlePaddle framework
- ****Status:**** Active, "influence is huge, extended projects numerous"

****Marker / Surya****

- ****Strengths:**** Supports >90 languages; layout detection, reading order
- ****Weaknesses:****
 - > "Marker supports a limited number of languages, and Figure processing is not perfect"
- ****License:**** Not stated
- ****Hardware Requirements:**** Not stated

****Unstructured****

- ****Strengths:**** Multiple document formats; ETL-focused
- ****Weaknesses:**** Limited layout analysis capability (marked with "?" in comparison table)
- ****License:**** Not stated
- ****Status:**** Commercial backing (\$25M funding); enterprise version available

****Gptpdf, Zerox, Chunkr, pdf-extract-api, Sparrow****

- ****Table Handling:**** Article notes these multimodal approaches show "poor performance" on complex tables
- ****Licenses:**** Not disclosed
- ****Note:**** Author states "Sparrow is only good at handling common formalized documents"

Commercial Service Findings (Author Testing)

Author tested 5 paid services on complex merged-cell tables:

1. ****TextIn:**** Tables recognized; "distinction of the content within the table is not obvious"
2. ****PDFLuxe:**** "Lost table title"
3. ****Mathpix:**** "Recognition is correct, but there are no merged cells"
4. ****Doc2x:**** "Recognition is correct, and the content is separated by spaces"
5. ****Baidu:**** "Content correct but table structure lost"

Licenses & Requirements NOT Disclosed

The article **does not provide software licenses (GPL/MIT/Apache), model-weight licenses, GPU/VRAM specs, or maintenance status** for any tool except noting:

- MinerU: "High degree of completion"
- PaddleOCR: Active ecosystem
- gptpdf: "No longer maintained"

Recommendation for Financial Tables

> "For complex tables, PDFflux or TextIn are more reliable. If you want to build your own document parsing solution, use MinerU as a base, and use the PaddleOCR toolkit to train the corresponding models."

Conclusion: Article confirms all open-source tools struggle with dense, multi-column financial tables; commercial services perform better but imperfectly.

5. assistant (2026-05-31T04:18:13.277Z)

The fetch succeeded. This is a roundup/practitioner article. Let me assess the source quality and extract the falsifiable claims.

This is a blog post by an individual author (Meursault / liduos.com), combining a third-party benchmark (Guangzhou Institute of Software Application Technology) with the author's own testing of commercial services. It is a secondary roundup, not primary institutional research ? best classified as "blog."

Key observations bearing on the research question:

- It explicitly does NOT report any license information (no GPL/MIT/Apache, no model-weights licenses) ? so it provides no help on muneem's hard license-verification constraint.
- It provides qualitative claims on table-handling reliability for several local tools (MinerU, PaddleOCR, Marker/Surya), which is directly relevant.
- The central finding ? open-source tools generally struggle with complex/multi-column tables while paid services do better ? is highly relevant to the VLM-vs-pipeline-vs-cloud question.

Now I'll return the structured output.

6. user (2026-05-31T04:18:20.435Z)

Structured output provided successfully

7. assistant (2026-05-31T04:18:29.663Z)

I've completed the source extraction. Summary of what I found:

Source: <https://liduos.com/en/ai-developer-tools-series-2-open-source-document-parsing.html> ? a December 8, 2024 blog roundup by an individual author (Meursault / ?????). It combines a third-party benchmark (Guangzhou Institute of Software Application Technology) with the author's own informal testing of five commercial services. Classified as **blog** (secondary roundup, not primary institutional benchmarking).

Relevance to muneem's research question: Moderate, with one important gap.

- **Useful:** It directly supports the thesis that open-source OCR/document-parsing tools generally struggle on complex multi-column tables (muneem's exact pain point with Axis/borderless statements), and that MinerU + PaddleOCR are the practitioner-favored local build foundation. It gives specific table-handling weaknesses for MinerU (row/column errors on complex tables, resource-heavy) and Marker/Surya (limited languages, imperfect figures).
- **Critical gap for muneem:** The article provides **zero license information** (no code license, no model-weights license) and **no GPU/VRAM specs** for any tool. So it cannot help verify the hard commercial-license-safety constraint (the CC-BY-NC / GPL / AGPL concerns around LayoutLMv3, Surya, Marker, etc.) ? those must be sourced elsewhere.

I returned 5 falsifiable claims (2 central, 3 supporting), each with a verbatim quote, via the StructuredOutput tool.